10. Distributed Rate Limiter

> **Interviewer:** Design a rate limiter for our API platform. Something like "a user can call `/login` 5 times a minute, and 100 requests a minute overall per API key." It has to work correctly even though we run **hundreds of gateway nodes** behind a load balancer.

Candidate: This is a fun one because the "hard part" isn't the business logic — it's a single number (a per-client counter) that hundreds of machines need to agree on, at very low latency, without talking to each other directly.

---

1. Clarifying the requirements

> **Candidate:** A few questions before I design: 1. What's the unit we're limiting — per user, per API key, per IP, or several at once? 2. Do different endpoints get different limits (e.g. `/login` stricter than `/search`)? 3. Do we need **bursts** tolerated, or is it a hard, smooth cap? 4. How exact does this need to be — "at most ~105/min slipped through" acceptable, or provably never-over-100? 5. What does a rejected request get back — just `429`, or `Retry-After` + remaining quota?
>
> **Interviewer:** Per API key, with per-endpoint overrides. Yes, tolerate short bursts — that's normal client behavior. Approximate is fine as long as it's not wildly off — this is abuse prevention, not a financial ledger. And give them `Retry-After` and remaining quota so well-behaved clients can back off politely.

Candidate: Good — "approximate is fine" opens the door to much cheaper designs than "must never be off by one," which is the burger-giveaway problem. Requirements:

Functional - `check(apiKey, endpoint)` → `ALLOW` or `DENY`, called on (almost) every incoming request. - Configurable limits per `(apiKey tier, endpoint)`, e.g. 100 req/min default, 5 req/min on `/login`. Tolerate short bursts above the steady-state rate. - On deny: return `429` + `Retry-After` + remaining quota.

Non-functional — this is where it gets interesting - **Latency:** this check sits in front of *every* request — must add microseconds/low single-digit milliseconds, never become the slow part of the request. - **Global correctness under massive fan-in:** hundreds of gateway nodes deciding for the *same* client — the limit must hold in aggregate, not per node, else "100/min" quietly becomes "100/min × N nodes." - **Availability:** the limiter itself must never be why the whole API goes down. - **Horizontal scale:** must handle the same request volume as the API layer it protects.

---

2. Estimation

> **Candidate:** Let me size the fan-in, because that's what decides where the counter lives.

```
Platform: 500,000 req/sec at peak, spread across 200 gateway nodes.
  -> each node individually sees ~2,500 req/sec. Trivial for one node.
  -> but they're all deciding limits for an overlapping pool of API keys.

Distinct API keys: ~50M. One counter per (key, endpoint, window).
  50M * ~100 bytes (key name + int + TTL bookkeeping) ≈ 5 GB.
  -> fits in memory easily, no disk needed.

If EVERY check is a network round trip to a shared store:
  500k checks/sec, each ~0.5-1ms round trip to Redis in the same AZ.
  A single Redis node does ~100k+ ops/sec -> need a handful of shards
  (5-10) to cover 500k/sec comfortably.

=> Two real design questions: WHERE does the counter live (so 200 nodes agree)?
   And HOW do we avoid paying a network hop on every request, given 500k/sec
   is a lot of hops even at sub-ms latency?
```

Candidate: So this isn't a storage-capacity problem — 5 GB is nothing. It's a **concurrency + latency** problem: hundreds of writers hammering shared counters, and a latency budget measured in microseconds.

---

3. V1 — a single node, in-process (and why it breaks immediately)

> **Interviewer:** Start simple. What's the naive version?

Candidate: If I had *one* gateway process, I'd keep an in-memory map `Map<apiKey, count>`, reset every minute — cheapest possible check, no network call at all.

```
   [ Client ] --request--> [ Single gateway process ]
                                in-memory: counts[apiKey]++
                                if counts[apiKey] > limit -> 429
```

Why it breaks the moment we scale out: the instant we run **N gateway nodes** behind a load balancer (required for 500k req/sec), each node has its *own* map. A client's traffic spreads across all N nodes, and each independently thinks "you're under 100, allowed" — the client can now get up to `100 × N` requests through. **The moment state isn't shared, the limit isn't real.**

---

4. The distributed version — a shared counter, and the race it creates

> **Interviewer:** Right, so make the state shared. What's your first cut?

Candidate: Move the counter out of each node's memory into a **shared, fast store** that every gateway node talks to — Redis. Naive version:

```
GET  ratelimit:{apiKey}:{windowMinute}      -> count
if count < limit:
    INCR ratelimit:{apiKey}:{windowMinute}
    return ALLOW
else:
    return DENY
```

> **Interviewer:** I see a bug already.

Candidate: Yes — **read-then-write across two round trips**, and with hundreds of nodes hitting the same key concurrently, there's a race window between the `GET` and the `INCR`.

```
Node A: GET count -> 99   (limit 100, thinks "1 slot free")
Node B: GET count -> 99   (same instant, also thinks "1 slot free")
Node A: count < 100 -> INCR -> allow request #100
Node B: count < 100 -> INCR -> allow request #101   <-- oversold!
```

Two nodes both read "99," both conclude "I have room," both increment. The limit of 100 just became 101 — and at 500k/sec with 200 nodes, this race fires constantly, not rarely. **The check and the increment must be a single atomic operation**, not two round trips with a gap a second node can slip into.

---

5. Closing the race — atomic check-and-increment with a Redis Lua script

> **Interviewer:** So how do you make "check and increment" atomic across every node in the fleet?

Candidate: Push the whole "read, compare, increment" sequence **into Redis itself** as one Lua script via `EVAL`. Redis executes an entire Lua script **single-threadedly** — no other client's command, from any node, can interleave in the middle of it.

```
EVAL "
  local count = redis.call('GET', KEYS[1])
  if count == false then
    redis.call('SET', KEYS[1], 1, 'EX', ARGV[2])
    return 1                        -- first request in this window: allow
  end
  count = tonumber(count)
  if count < tonumber(ARGV[1]) then
    redis.call('INCR', KEYS[1])
    return 1                        -- allowed
  else
    return 0                        -- denied
  end
" 1 ratelimit:apiKey123:min 100 60
```

Because the whole block runs as one indivisible step *inside Redis*, node A's and node B's scripts are strictly ordered — whichever arrives first sees `count = 99` and increments to 100; the other sees `count = 100`, which is `not < 100`, and is denied. **No window exists where both can succeed.** This is the centerpiece of the whole design: the atomicity lives in the data store, not in application code, precisely because application code can't coordinate across 200 independent processes without paying for a distributed lock — Redis already gives us that coordination for free, per key, at memory speed.

Why not a distributed lock (e.g. Redlock) around a plain `GET` / `SET`? It would work, but it's strictly worse: you pay for lock acquisition + release (extra round trips) to protect two commands, when Redis can already do the equivalent work as one server-side script with zero extra round trips. Locking is for when you need multi-step logic across *multiple* keys/stores; here it's one key, so let Redis's own single-threaded execution be the lock.

---

6. Algorithm choice — fixed window vs sliding log vs sliding counter vs token bucket

> **Interviewer:** You picked a fixed 60-second window in that script. Walk me through the alternatives — I want to know why you'd pick one over the others.

Candidate: This is the crux of the *product* decision, separate from the atomicity question. Four contenders:

```
FIXED WINDOW                          SLIDING WINDOW LOG
|0:00        0:59|1:00     1:59|      [t1][t2][t3]...[tN]  <- one
     ^100 reqs here   ^100 more here     timestamp PER request, exact
     = 200 reqs in a 2s span at the      count, but O(requests) memory
     boundary (the classic burst bug)    -> too much RAM at our scale

SLIDING WINDOW COUNTER (hybrid)       TOKEN BUCKET
est = curr + prev*(1 - elapsed/60)    [#####] full -> take 1 -> [####.]
keep prev+curr window counts, weight  refills continuously at `rate`
by how far into curr window we are.   tokens/sec, capped at `capacity`.
cheap, smooths the boundary bug,      naturally allows a burst up to
no real burst allowance.              capacity, throttles after that.
```

Algorithm	Memory	Accuracy	Burst handling	Verdict
Fixed window	O(1)/key	Poor — up to 2x burst at the boundary	None	simplest, real bug
Sliding window log	O(requests)	Exact	Exact	too much memory at 500k/sec
Sliding window counter	O(1)/key	Good approximation	Good	default, no burst allowance
Token bucket	O(1)/key	Good	**Models bursts naturally**	**my pick**

My choice: token bucket. Bursts should be tolerated, and that's exactly what a token bucket is *for* — a client saves up to `capacity` tokens during quiet periods and spends them in a quick burst, then throttles to the steady refill rate. Fixed window is rejected for the boundary-doubling bug (100 at 0:59 + 100 at 1:00 = 200 in 2 seconds). Sliding window log is rejected because a timestamp per request is exact but O(requests) in memory at 50M keys × high QPS — too much RAM for precision we don't need. Sliding window counter is the honorable runner-up for smooth approximation without burst allowance.

The Lua script for token bucket does the same atomic trick with a bit more math in one `EVAL`: compute elapsed time since last refill, add `elapsed * rate` tokens capped at `capacity`, then if `tokens >= 1`, subtract 1 and allow.

---

7. Where does the limiter live, and cutting latency in front of the shared counter

> **Interviewer:** Where does this code actually run, relative to the gateway nodes?

Candidate: Three placements: **(A) a plugin inside the L7 API Gateway** — my default; **(B) a sidecar** per node over local IPC; **(C) a standalone central rate-limit service** over the network. All three ultimately call the same atomic Redis script; the difference is how far the *first* hop is. **Why A:** the limiter needs the parsed request (API key, endpoint, headers), and only L7 has already parsed that — reusing that context avoids an extra hop, and rejecting happens **before** the request is proxied to a backend, so a denied request never costs a backend thread. **Why not C:** a standalone service adds a full network round trip *in addition to* the Redis round trip, for no benefit. **B** is the reasonable middle ground for independent deployment/versioning (service-mesh style) — same-host IPC is cheap.

> **Interviewer:** Every one of your 500k checks/sec still does a Redis round trip — 0.5-1ms each, on the hot path of every request. Can you do better?

Candidate: This is exactly the tension from the estimation step.

```
Option A — fully centralized (every check hits Redis):
  + always globally accurate, atomic Lua script guarantees no oversell
  - every request pays a real network round trip

Option B — fully local (each node keeps its own bucket, no shared state):
  + zero extra latency, pure in-process math
  - N nodes each give the client the FULL limit -> effective limit
    becomes limit * N. This is V1's bug again, just dressed up.

Option C — hybrid: local cache of "tokens remaining" ◄──refill ~100ms──► Redis (truth),
  lazily refreshed; the ACTUAL decrement still round-trips to the atomic
  Lua script when the local cache says "close to empty."
```

My recommendation: default to Option A. 0.5-1ms is small relative to typical API response times, and Redis co-located in the same AZ/rack keeps that hop cheap. I'd only move to **Option C** if profiling showed the Redis round trip was the *actual* bottleneck — and I'd say the tradeoff out loud: local caching means a node can allow **slightly more than the global limit** for a brief window, deciding off a slightly

stale local view. That's the **exact vs. approximate limiting** conversation: Option A is exact, Option C trades a bit of over-admission for a big latency win. Given "approximate is fine" was already agreed, Option C is legitimate here — I'd just get product sign-off on "up to ~X% overage during bursts."

---

8. Sharding Redis so it doesn't become the bottleneck (or SPOF)

> **Interviewer:** You've centralized everything onto Redis. Now Redis is a single hot dependency for 500k checks/sec. What's your plan?

Candidate: Two separate concerns: **throughput** and **single point of failure**.

Throughput — shard by `apiKey` hash across a Redis Cluster:

```
apiKey hash ──► [shard0][shard1][shard2]...[shard9]   each shard holds
picks a shard    a slice of the 50M keys, ~50k/s each — 500k/sec spread
                  across 10 shards is comfortably inside a single
                  Redis node's ~100k+ ops/sec capability.
```

A client's key lives on one shard (needed since the atomic Lua script operates on a single key), but *different* clients spread across the whole cluster, so no shard sees the full 500k/sec. **Hot key risk:** one enormous partner key could still dominate a shard — give it a dedicated shard, or push it to the Option-C local-cache hybrid.

Availability/SPOF — replicate each shard (primary + replicas). Writes must go to the primary; reading a stale replica for check-and-increment would reintroduce the section-5 race, since two nodes could both see "quota available" off lagging replicas. On failover, an in-flight counter can reset — accepted risk (more in section 13); the limiter's job is stopping *sustained* abuse, not being a perfectly precise ledger.

Why not a SQL counter table instead of Redis? `SELECT ... FOR UPDATE` on one row still serializes every writer behind a lock, and disk-backed commits add an order of magnitude of latency we don't have budget for at 50k+/sec per key — plus we'd need a cron job to clean up expired-window rows that Redis's `EXPIRE` gives for free. SQL earns its keep when you need joins, multi-row ACID, or durability for data that matters tomorrow; a counter meaningless a minute from now needs none of that.

---

9. Multi-region

> **Interviewer:** We're now global — gateways in the US and in the EU. Same client can hit both. One global limit, or per-region?

Candidate: Per-region counters, not one global counter.

```
Region: US  [Gateway nodes] -> [Redis-US]   Region: EU  [Gateway nodes] -> [Redis-EU]
             (independent counters, independent decisions per region)

vs. ONE global counter (cross-region Redis/DB): every check pays 50-150ms
cross-region latency -> blows the microsecond/low-ms budget from section 2.
```

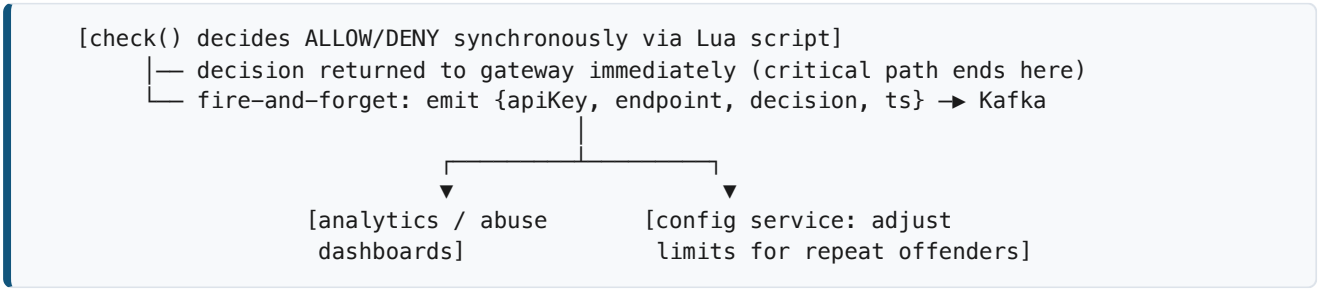
A client's "100 req/min" effectively becomes "100/min *per region it happens to hit*" — one splitting traffic across both regions could get ~2x the intended limit. That's a **deliberate tradeoff**: perfect global accuracy needs a synchronous cross-region hop on the hot path, defeating the point of a rate limiter. If accuracy mattered more, mitigate with sticky per-key routing to one "home" region, or asynchronously aggregate per-region counts into a global view for **monitoring/alerting** while still enforcing the looser real-time limit.

---

10. Async — what moves off the hot path

> **Interviewer:** Anything here that shouldn't be synchronous?

Candidate: The decision itself (`ALLOW` / `DENY`) has to be synchronous — the gateway can't proxy the request without knowing the answer. But everything *around* it can be async:



- **Rule/config lookups** (which limit applies to this tier + endpoint) are cached locally on every gateway node, refreshed on a slow interval (seconds) — never looked up synchronously per request. Config changes rarely; checks happen 500k times a second, so we optimize the read path and accept a few seconds of staleness on limit changes.
- **Abuse analytics/alerting** (e.g. "this key was denied 10k times in 5 minutes, maybe block it entirely") consumes the denial stream from Kafka asynchronously — never on the check path.

---

11. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Counter store	**Redis (Cluster, sharded by apiKey)**	In-memory, sub-ms `INCR` / `EVAL`; single-threaded execution gives free atomicity. *Not Postgres/MySQL row counter* — a locked row serializes every writer and disk commits add ~10x the latency we can afford at 50k+/sec per key.
Atomicity mechanism	**Redis Lua script (`EVAL`)**	Runs "read, compare, increment, set TTL" as one indivisible step — no other command can interleave mid-script. *Not separate*

Concern	Choice	Why this, and why not the alternative
		`GET` + `INCR` — reintroduces the read-then-write race (section 4). *Not a distributed lock (Redlock)* — pays extra round trips to protect two commands Redis already serializes for free.
Algorithm	**Token bucket** (sliding window counter as fallback)	Naturally tolerates short bursts while capping sustained rate; O(1) memory/key. *Not fixed window* — boundary-doubling bug. *Not sliding window log* — O(requests) memory is wasteful at 50M keys × high QPS.
TTL / cleanup	**Redis native `EXPIRE` / `SET ... EX`**	A minute-window counter vanishes after a minute for free. *Not a SQL cron cleanup job* — extra moving part Redis does natively.
Where the check runs	**In-process plugin inside the L7 API Gateway**	Reuses the gateway's parsed request context; rejects before a backend thread is spent. *Not a standalone microservice* — adds a full extra hop for no benefit.
Latency mitigation	**Direct Redis check by default; local-cache hybrid only if profiling demands it**	0.5-1ms same-AZ round trip is fine relative to response time. *Local-only counters* rejected outright — multiplies the effective limit by N, the exact bug that started this design.
Multi-region	**Per-region Redis clusters, independent limits**	Keeps the hot-path round trip local. *Not one global cross-region counter* — 50-150ms round trip is a non-starter.
Async abuse signal path	**Kafka stream of decisions**	Durable, replayable, decouples analytics from the synchronous decision path. *Not a synchronous analytics call* — would add a non-critical dependency to the hot path.
Config/rules storage	**Small SQL table/config service, cached on every gateway node**	Low write volume, read-heavy, tolerant of staleness. *Not looked up per-request* — adds a synchronous dependency to every check.

The one-sentence justification you'd say out loud: "*Redis is the right store because it's in-memory, gives me native TTLs, and — critically — lets me express check-and-increment as one atomic Lua script so hundreds of gateway nodes never race each other on the same counter; I default to a token bucket for burst tolerance, shard Redis by client key to scale throughput and contain hot keys, keep regions independent to avoid a cross-region hop on the hot path, and only reach for local per-node caching if profiling proves the Redis round trip itself is the bottleneck — accepting a small, explicit amount of over-admission in exchange for latency.*"

---

12. Consistency model — what's strong, what's eventual

- **Strong (within a shard/region):** the `ALLOW` / `DENY` decision and the counter increment, linearized by Redis's single-threaded Lua execution — no two nodes can both win the "last slot."
- **Eventual/approximate, and that's fine:** the sliding-window-counter's weighted estimate; the local-cache hybrid's "tokens remaining" under Option C; the per-region split (a client can get ~2x by splitting traffic across regions); the async analytics/alerting stream.
- **Why this split is correct here:** a rate limiter's contract is "prevent sustained abuse," not "produce a perfectly audited ledger" — the opposite of the burger-giveaway/flash-sale problems where

oversell was catastrophic and had to be exact. Being off by a handful of requests during a regional split or brief failover is an acceptable, explicit cost for a much simpler and faster system.

---

13. Failure modes & graceful degradation

> **Interviewer:** The one I really want to hear: what happens when Redis is unreachable?

Candidate: The single most important question in this problem, so let me not dodge it.

```
Redis unreachable/timeout during a check:
  FAIL OPEN  -> allow the request.
    + availability: a Redis blip doesn't take down the whole API
    - defenseless against abuse exactly when things are already fragile
  FAIL CLOSED -> deny the request.
    + safety: never lets unlimited traffic through
    - Redis becomes a single point of failure for 100% of legitimate
      traffic too — the limiter itself takes down the API it protects
```

My recommendation: fail open, but with a short-lived local fallback bucket — while Redis is unreachable, each gateway node falls back to a conservative in-process token bucket using the last known limit (Option B from section 8, deliberately accepted here as a *temporary* degraded mode, not the steady state). That bounds the damage — every node still enforces *something* — without letting one dependency's outage cascade into total API exposure. Pure fail-closed is only justified when the protected resource is fragile enough that *any* unbounded traffic is worse than blocking everyone — e.g. shielding a metered payment provider from real overage charges.

Failure	What happens	Mitigation
Redis shard down	Its clients' checks fail	Replica promotion; fail-open + local fallback bucket bridges the gap
Redis primary failover	Brief counter reset possible	Accepted risk — a few seconds of "fresh" quota is cheap compared to full outage
Lua script bug/timeout	Check hangs or errors	Treat identically to "Redis unreachable" → same fail-open fallback path, never a silent hang
One gateway node dies	No rate-limit state lost	State lives in Redis, not the node; LB just stops routing to it
One region's Redis cluster down	That region's checks degrade	Other region unaffected (independent clusters); local fallback bucket covers the gap
Hot API key saturates one shard	That shard slows for everyone sharing it	Dedicated shard for the hot key, or route it to local-cache hybrid

---

14. Follow-up curveballs

> **Interviewer:** How would you rate-limit by IP *and* by API key at the same time?

Candidate: Two independent atomic checks (two `EVAL` calls, or one script touching two keys), and the request must pass **both** — most-restrictive-wins. Each check stays its own atomic operation on its own key so I never need a multi-key transaction across shards.

> **Interviewer:** How do you let a client see their remaining quota without consuming it?

Candidate: A separate read-only `peek` path — a plain `GET` on the counter/bucket key, never routed through the decrement script. Slightly stale by definition, which is fine for an informational "X requests remaining" header.

> **Interviewer:** Marketing/ops wants a live dashboard of "top rate-limited clients right now." How do you build that without slowing down the hot path?

Candidate: Never read the hot counters for this. Consume the async Kafka decision stream (section 11) and aggregate denials per client on a rolling window downstream — same principle as the burger-giveaway's "burgers remaining" counter: keep cosmetic reads away from the keys gating real traffic.

> **Interviewer:** A client complains they're rate-limited despite being under 100 req/min by their own count. How do you debug it?

Candidate: Likely culprits: they're counting only 2xx responses while our counter increments on every attempt including their own retries; clock/boundary confusion if we're on fixed window (another mark against it); or they share one API key across multiple services that each burn quota independently. I'd pull the Kafka decision stream for their key and show the actual `(timestamp, endpoint, decision)` sequence — the async audit trail earns its keep.

---

15. Deep-dive: "Why not just keep the counters in each server's local memory — no shared store at all?"

> **Interviewer:** Redis is one more thing to run, patch, and page on. Why not skip it entirely — keep the counter as a plain in-memory map on each gateway node, like your V1? It's faster and it's one less dependency.

Candidate: I actually raised that as V1 in section 3 and rejected it for a specific reason, but it's worth being precise about *why*, because "just keep it local" comes back in a few disguises and not all of them are equally bad.

The core problem: N nodes, N× the limit

```
limit = 100 req/min per API key.  Fleet has N = 200 gateway nodes behind an L4/L7 LB.
A single client's traffic is round-robined/hashed across MANY of those 200 nodes.

Node 1: sees 100 of the client's requests -> local counter hits 100 -> starts denying
Node 2: sees 100 of the SAME client's requests -> its OWN counter hits 100 -> denies
Node 3: ... same ...
  ...
=> the client can get up to 100 * (number of nodes it happens to land on) through,
   while every individual node honestly believes it enforced "100/min."
```

Nothing is buggy about any single node's logic — the map, the increment, the comparison are all correct *locally*. The failure is architectural: **the limit is a property of the client, but the state enforcing it is partitioned by node**, and the load balancer's job is precisely to spread one client across many nodes. The more nodes you add for scale, the *worse* the effective limit gets — which is backwards for a system meant to protect capacity.

Three real options, not two

How it works	Each node keeps its own `Map<apiKey, count>`; never talks to peers	Every check is an atomic round trip to shared Redis (section 5)	Each node counts locally, then gossips/broadcasts its local count to peers every ~100-500ms; each node sums local + last-known-peer counts for a global estimate
Per-check latency	~0 (in-process)	+0.5-1ms network round trip	~0 (in-process); sync happens off the request path
Effective limit for one client	up to `limit × N` — unbounded as fleet scales	exactly `limit` (atomic, section 5)	`limit + (N−1) × rate × syncInterval` — bounded overshoot, not unbounded
Accuracy	Broken — not a real limit	Exact	Approximate, tunable via sync interval
Availability / SPOF	No shared dependency — can't go down	Redis outage affects every node (mitigated in section 13/16)	No single point of failure — a dead peer just means a slightly staler view
Scales with fleet size	Gets *worse* as N grows	Unaffected by N (shards, not replicates, the work)	Overshoot grows mildly with N, but stays bounded, not multiplicative
Complexity	Trivial	Low (one Lua script)	Higher — needs a gossip/broadcast channel (e.g. Redis Pub/Sub, a mesh, or CRDT-style counters) and reconciliation logic
Verdict	Rejected — this is V1's bug, restated	**My default** (sections 4-8)	A legitimate fallback/degraded mode, not a primary design

The third option is worth naming explicitly because it's the honest middle ground: it's what you'd reach for if you wanted zero hot-path network calls *and* better-than-local accuracy — e.g. a CRDT-style `G-Counter` per node, merged via periodic gossip. It never becomes exact (there's always a sync-interval's worth of lag baked into the bound), but the overshoot is *bounded and tunable*, unlike pure local memory where overshoot scales linearly with however many nodes you happen to add. In this design it's exactly the fail-open fallback bucket from section 13 (detailed further in section 16) — a temporary, explicitly degraded mode when Redis is unavailable, not the steady-state design, because "approximate but bounded" is worse than "exact" whenever exact is affordable — and at 0.5-1ms per check, it is.

> **Rule of thumb:** *If N independent processes can each independently decide "yes" for the same client, the real limit is `stated limit × N`, no matter how correct each process's local logic is. Shared state isn't an optimization here — it's the only way the limit means what it says.*

---

16. Deep-dive: Redis is slow, not down — fail-open vs. fail-closed, and the race that reappears under load

> **Interviewer:** Section 13 covered Redis being *unreachable*. Here's the meaner version: during a traffic spike, Redis isn't down — it's just **slow**, p99 climbing from 1ms to 300ms. Do you fail open or fail closed? And separately — walk me through, precisely, what goes wrong if the check-and-increment isn't atomic under exactly this kind of load.

Candidate: Two related but distinct questions — let me take them one at a time, because conflating them is how people ship the wrong fix.

Part 1 — why "slow" is worse than "down"

A hard failure (connection refused) is easy: fail fast, fall back immediately. **Slow is the dangerous case**, because every gateway thread/connection waiting on that 300ms Redis call is a thread that's *not* serving other requests. Under a spike, that's how one degraded dependency turns into a full gateway outage:

```
Normal:   [gateway thread] --0.5ms--> [Redis] --0.5ms--> back, thread freed instantly

Degraded: [gateway thread] --...300ms...--> [Redis] --...--> back
           meanwhile 500k req/sec keep arriving, each grabbing a thread/connection
           -> connection pool exhausts -> EVEN CHECKS FOR HEALTHY, UNTHROTTLED
              keys now queue behind the pool -> the rate limiter, whose whole job is
              protecting the API, becomes the reason the API falls over.
```

This is why a **short, aggressive timeout + circuit breaker** on the Redis call matters more than the fail-open/closed choice itself: without one, "slow" silently becomes "down for everyone," including clients who were nowhere near their limit.

Part 2 — fail open vs. fail closed, decided explicitly

```
FAIL OPEN  (allow when Redis can't answer in time)
  + the API stays up; a dependency blip doesn't cascade into an outage
  - zero abuse protection for the exact window when things are already stressed

FAIL CLOSED (deny when Redis can't answer in time)
  + never lets unbounded traffic through
  - the rate limiter -- a *protective* layer -- now takes down 100% of legitimate
    traffic on a Redis blip. The cure becomes worse than the disease.
```

My recommendation, same conclusion as section 13, with the mechanism spelled out: fail open, gated by a circuit breaker, dropping to a **local in-process token bucket** as the fallback — not "no limiting at all." Concretely:

```
while Redis p99 < threshold (e.g. 20ms):     NORMAL — every check hits the atomic
                                               Lua script, exact global limit.

circuit breaker trips (error rate / latency  DEGRADED — stop calling Redis entirely
over threshold for a short window):           (don't even try-and-timeout per request,
                                               that's still 500k slow calls). Each node
                                               falls back to its OWN local token bucket,
                                               seeded from the last known limit.

circuit breaker half-opens, probes Redis,    RECOVERY — a trickle of real checks resume;
latency back to normal:                       on success, flip back to NORMAL.
```

During DEGRADED, we've knowingly re-entered section 15's "local memory only" world — but **deliberately, temporarily, and boundedly**, not as the steady state. Every node still enforces *something* close to the real limit instead of either 0 (fail-closed cascading) or ∞ (naive fail-open with no local guard). That's the graceful-degradation answer product actually wants: abuse protection degrades gracefully under a dependency outage, rather than being binary in lockstep with the outage.

Exception — when fail-closed is still correct: if the protected resource meters real money per call (e.g. we're the layer in front of a paid upstream billed per request), letting traffic through unmetered during a Redis blip could mean real financial exposure worse than an outage. That's a case-by-case call with the business, not a default.

Part 3 — the read-modify-write race, re-derived under exactly this load

> **Interviewer:** Now the race. Walk me through it precisely.

Candidate: Same shape of bug as section 4, but worth re-deriving here because it's precisely *under spike load* — when Redis latency is climbing and concurrency is highest — that the naive, non-atomic version fails most often. "Just add Redis" isn't automatically a fix if it's implemented as two commands instead of one:

```
limit = 100.  Two gateway nodes, same apiKey, same instant, NAIVE (non-atomic) version:

  Node A:  GET ratelimit:key  -> 99      (thinks: "1 slot free")
  Node B:  GET ratelimit:key  -> 99      (same instant, ALSO thinks: "1 slot free")
  Node A:  99 < 100  -> INCR -> 100      -> ALLOW  (request #100 — correct)
  Node B:  99 < 100  -> INCR -> 101      -> ALLOW  (request #101 — OVERSOLD)

  Both read the SAME stale value before either write lands. The gap between the
  GET and the INCR is exactly wide enough for a second node to slip through it.
  At 500k/sec across 200 nodes, this isn't a rare accident -- it fires continuously,
  and gets MORE likely the slower/busier Redis is, since more requests are in
  flight for the same key at once.
```

The atomic fix (recap + why it holds even here): push check-and-increment into Redis as one Lua script (section 5) — Redis executes the *entire script* single-threaded, so node A's and node B's scripts are strictly serialized, never interleaved:

```
EVAL "
  local count = tonumber(redis.call('GET', KEYS[1]) or '0')
  if count < tonumber(ARGV[1]) then
    redis.call('INCR', KEYS[1])
    redis.call('EXPIRE', KEYS[1], ARGV[2])   -- only matters on first write
    return 1                                  -- ALLOW
  else
    return 0                                  -- DENY
  end
" 1 ratelimit:apiKey123:min 100 60

Node A's script runs fully: sees 99, increments to 100, returns ALLOW.
Node B's script runs fully AFTER (never interleaved): sees 100, 100 < 100 is
false, returns DENY.  There is no separate read step visible to another client
mid-script -- so there is no window for two nodes to share the same stale read.
```

Why the race gets worse, not better, exactly when Redis is slow: higher latency means more concurrent in-flight requests are all mid-flight at once for the same hot key — a wider window for a *naive* two-step version to interleave. The atomic version doesn't care how slow or contended Redis is under load: the ordering guarantee (single-threaded script execution) holds regardless of how many scripts are queued — they just queue and run one at a time. That is exactly why the fix is "make it one atomic op," not "add a lock" or "retry harder" — those add latency and can still race on retry timing; the Lua script removes the race by construction, independent of load.

> **What to say out loud:** "*There are two independent failure modes here and I want to fix both, not pick one and hope. Correctness under concurrency is solved once, structurally, by making check-and-increment a single atomic Lua script — that holds no matter how many nodes or how much load is queued behind it. Availability under a slow dependency is solved by a circuit breaker that stops hammering a struggling Redis and drops every node to a local fallback token bucket — fail open, but not wide open. The one case I'd flip to fail-closed is when unmetered traffic has a real dollar cost downstream — that's a business call, not an engineering default.*"

Summary — the mental model

A distributed rate limiter is fundamentally **"hundreds of independent processes need to agree on one number per client, in well under a millisecond, without ever talking to each other directly."** The move that solves it: push the counter into a **shared in-memory store (Redis)** and make check-and-increment **one atomic server-side operation (a Lua script)** so Redis's own single-threaded execution — not application-level locking — closes the race between concurrent nodes. **Token bucket** is the algorithm of choice when bursts should be tolerated (which they usually should); shard Redis by client key to scale throughput and contain hot clients; keep regions independent rather than paying a cross-region hop on the hot path; and when Redis itself is unreachable, **fail open with a conservative local fallback** rather than either taking down the whole API (fail closed) or going fully unprotected. The recurring theme versus the burger-giveaway/flash-sale problems: there, oversell was catastrophic and demanded exactness; here, the limiter's job is deterring *sustained* abuse, so a little approximation in exchange for speed and availability is not just acceptable, it's the correct tradeoff.